Lossless compression (I)
Huffman coding, Golomb, Rice and Tunstall codes, examples, …

Karel Fliegel
(fliegek@fel.cvut.cz)

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

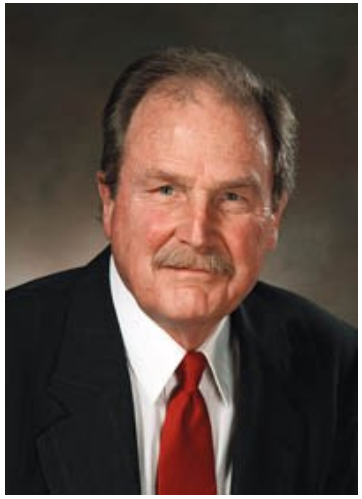
Technická 2
166 27 Prague 6
Czech Republic

27/2/2026

© Multimedia Technology Group

Contents

- **Shannon-Fano coding**
- **Huffman coding**
  - **Overview**
  - **Huffman** coding **algorithm**
    - **Minimum variance** and **canonical Huffman** codes
    - **Optimality** and **length** of **Huffman** codes
    - **Implementation** of **Huffman** codes
  - **Adaptive Huffman** coding
- **Golomb**, **Rice**, and **Tunstall codes**
- **Examples** of **Huffman coding**
  - Simple **lossless image compression**
  - Simple **audio signal lossless compression**



David Albert Huffman
(1925 – 1999)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Shannon-Fano coding: Overview

- **Shannon-Fano** coding
  - Predecessor of **Huffman** coding algorithm
  - **Claude Shannon** and **Robert Mario Fano** (1917 – 2016)
    - **Shannon–Fano tree** built to **define effective code table**
      - (**1**) **list** of **symbols** → **list** of **probabilities** or **counts**
      - (**2**) **sort** the **lists** of **symbols according** to **frequency** → **descending left** to **right**
      - (**3**) **divide list** into **two parts** → **close probabilities**
      - (**4**) **left part** of list **binary** 0 and **right part** is **binary** 1
      - **Recursively apply** steps (**3**) and (**4**) until **each symbol** has **corresponding code leaf**
    - **Shannon-Fano algorithm** is **not guaranteed** to generate **optimal code**
      - Almost **never used** → **Huffman coding** also **computationally simple** and produces **prefix codes** with **lowest possible expected codeword length**

Robert Mario Fano
(1917 – 2016)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

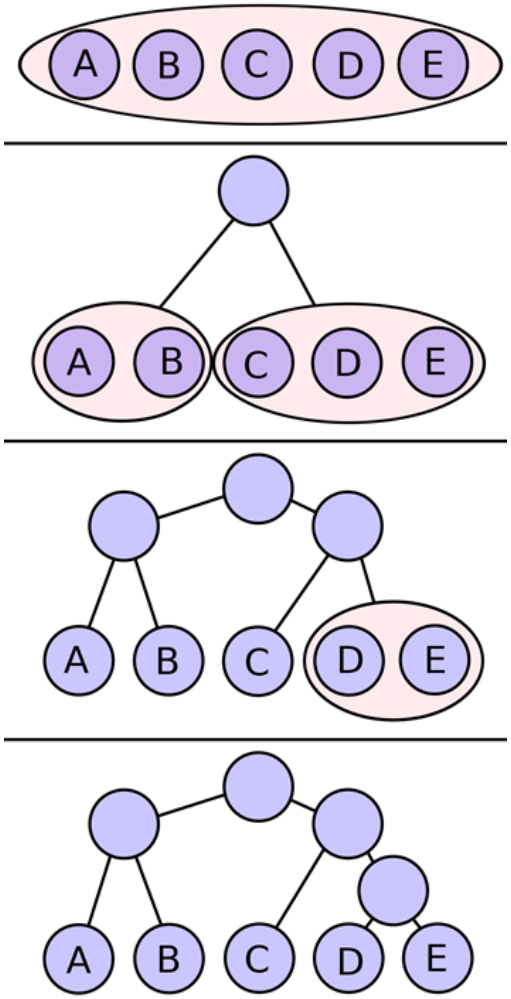
Shannon-Fano coding: Overview

Shannon-Fano coding

Example

- **Shannon–Fano tree** built to **define effective code table**
  - (**1**) **list** of **symbols** → **list** of **probabilities** or **counts**
  - (**2**) **sort** the **lists** of **symbols according** to **frequency** → **descending left** to **right**
  - (**3**) **divide list** into **two parts** → **close probabilities**
  - (**4**) **left part** of list **binary** 0 and **right part** is **binary** 1
  - **Recursively apply** steps (**3**) and (**4**) until **each symbol** has **corresponding code leaf**

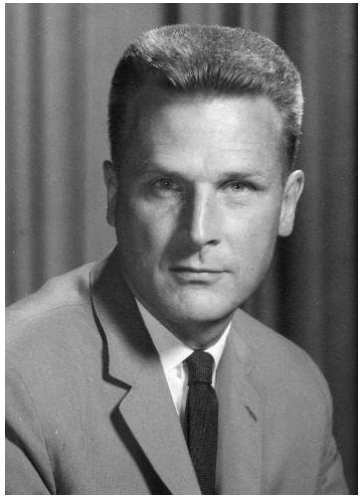
Symbol	A	B	C	D	E
Probabilities	0.385	0.179	0.154	0.154	0.128
First division	0		1		
Second division	0	1	0	1	
Third division				0	1
Codewords	**00**	**01**	**10**	**110**	**111**



By Andreas.Roever, SVG: Marlus_Gancher - drawn based on Datei:ShannonCodeAlg.png (bitmap version), GFDL, https://commons.wikimedia.org/w/index.php?curid=13058778

Huffman coding: Overview

- **Huffman** coding
  - Very **popular** coding algorithm
  - Utilization for **image**, **audio** and **text**, ...
    - Building Huffman codes for **known source model probability**
    - Building Huffman codes for **unknown source statistics**
  - **David Albert Huffman** (1925 – 1999)
    - Technique **developed** in **MIT class assignment**
    - Famous **Robert Fano** class (first on information theory)
    - **Better code** than **Huffman supervisor's**
      i.e. **Shannon-Fano code**
  - **Huffman codes**
    - **Prefix codes** optimal
      for **given model** (**set** of **probabilities**)



David Albert Huffman
(1925 – 1999)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Huffman coding: Algorithm

Huffman code procedure

- Based on **two observations** on **optimum prefix codes**
  - (**1**) In **optimum** code **symbols** that **occur more frequently** (higher **probability** of **occurrence**) have **shorter codewords** than **less frequent symbols**
  - (**2**) In **optimum** code **two symbols** that **occur least frequently** have **same codeword length**
    - Comes from **requirements** on **prefix codes**
    - **Shorter codeword** cannot be **prefix** of **longer one**

- **Procedure** adds **simple requirement**
  - **Codewords corresponding** to **two lowest probability symbols** differ **only** in **last bit**
    - For two **least probable** symbols γ and δ in **alphabet** → $\mathbf{m} * 0$ is codeword for γ → then $\mathbf{m} * 1$ is codeword for δ
      - $\mathbf{m}$ is **string** of 1s and 0s
      - $*$ denotes **concatenation**

© Multimedia Technology Group

Huffman coding: Algorithm

Huffman code procedure

Example

- Designing **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$ with **probabilities** $P(a_1) = P(a_3) = 0.2$, $P(a_2) = 0.4$ and $P(a_4) = P(a_5) = 0.1$
  - Related **source entropy** $H = 2.122$ bit/symbol
- **Sorting** letters with **descending probability**

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a_1	0.2	$c(a_1)$
a_3	0.2	$c(a_3)$
a_4	0.1	$c(a_4)$
a_5	0.1	$c(a_5)$

- Two letters with **lowest probability** a_4, a_5 and their **codewords**

$$c(a_4) = \alpha_1 * 0$$
$$c(a_5) = \alpha_1 * 1$$

where α_1 is **binary string** and $*$ **concetanation**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Huffman coding: Algorithm

Huffman code procedure

Example

- Defining **new** (four letter) **alphabet** $A' = \{a_1, a_2, a_3, a'_4\}$ where a'_4 is **composed** of a_4 and a_5 with **probability** $P(a'_4) = P(a_4) + P(a_5) = 0.2$
- **Sorting** letters with **descending probability**

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a_1	0.2	$c(a_1)$
a_3	0.2	$c(a_3)$
a'_4	0.2	α_1

- Two letters with **lowest probability** a_3, a'_4 and their **codewords**

$$c(a_3) = \alpha_2 * 0$$
$$c(a'_4) = \alpha_2 * 1$$

where $c(a'_4) = \alpha_1$ is **binary string**

- **Then** $\alpha_1 = \alpha_2 * 1$ which means that

$$c(a_4) = \alpha_2 * 10$$
$$c(a_5) = \alpha_2 * 11$$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Huffman coding: Algorithm

Huffman code procedure

Example

- Defining **new** (three letter) **alphabet** $A'' = \{a_1, a_2, a'_3\}$ where a'_3 is **composed** of a_3 and a'_4, **probability** $P(a'_3) = P(a_3) + P(a'_4) = 0.4$
- **Sorting** letters with **descending probability**

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a'_3	0.4	α_2
a_1	0.2	$c(a_1)$

- Two letters with **lowest probability** a_1, a'_3 and their **codewords**

$$c(a'_3) = \alpha_3 * 0$$
$$c(a_1) = \alpha_3 * 1$$

where $c(a'_3) = \alpha_2$ is **binary string**

- **Then** $\alpha_2 = \alpha_3 * 0$ which means that

$$c(a_3) = \alpha_3 * 00$$
$$c(a_4) = \alpha_3 * 010$$
$$c(a_5) = \alpha_3 * 011$$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Huffman coding: Algorithm

Huffman code procedure

Example

- Again defining **new** (two letter) **alphabet** $A''' = \{a''_3, a_2\}$ where a''_3 is d **composed** of a'_3 and a_1, **probability** $P(a''_3) = P(a'_3) + P(a_1) = 0.6$
- **Sorting** letters with **descending probability**

Letter	Probability	Codeword
a''_3	0.6	α_3
a_2	0.4	$c(a_2)$

- Only **two symbols** left a_2, a''_3 **assigning codewords**

$$c(a''_3) = 0$$
$$c(a_2) = 1$$

- That **means** $\alpha_3 = 0$ and **codewords** are

$$c(a_1) = 01$$
$$c(a_3) = 000$$
$$c(a_4) = 0010$$
$$c(a_5) = 0011$$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Huffman coding: Algorithm

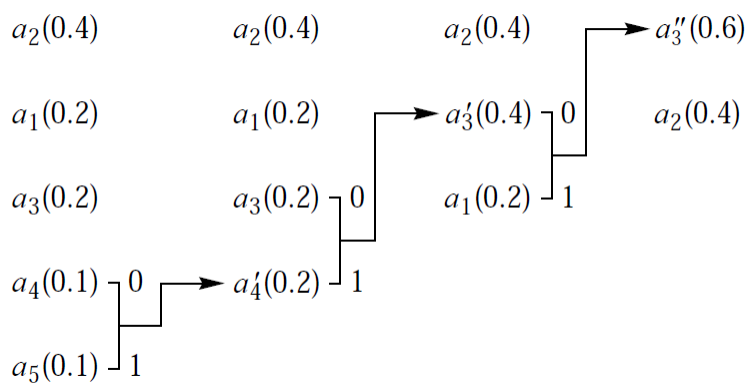
Huffman code procedure

Example

- Designing **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$ with **probabilities** $P(a_1) = P(a_3) = 0.2$, $P(a_2) = 0.4$ and $P(a_4) = P(a_5) = 0.1$

Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

- **Huffman encoding procedure** represented as **binary tree**



Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Huffman coding: Algorithm

Huffman code procedure

Example

- Designing **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$ with **probabilities** $P(a_1) = P(a_3) = 0.2$, $P(a_2) = 0.4$ and $P(a_4) = P(a_5) = 0.1$

Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

  – Related **source entropy** $H = 2.122$ bit/symol

  – Related **average Huffman code length**

$$l = 0.4 \cdot 1 + 0.2 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 4 + 0.1 \cdot 4 = 2.2 \text{ bit/symbol}$$

  – Related **redundancy** (**entropy** and average **length difference**)

$$0.078 \text{ bit/symbol}$$

  – **Redundancy** is **zero** for **probabilities** of **negative powers** of **two**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

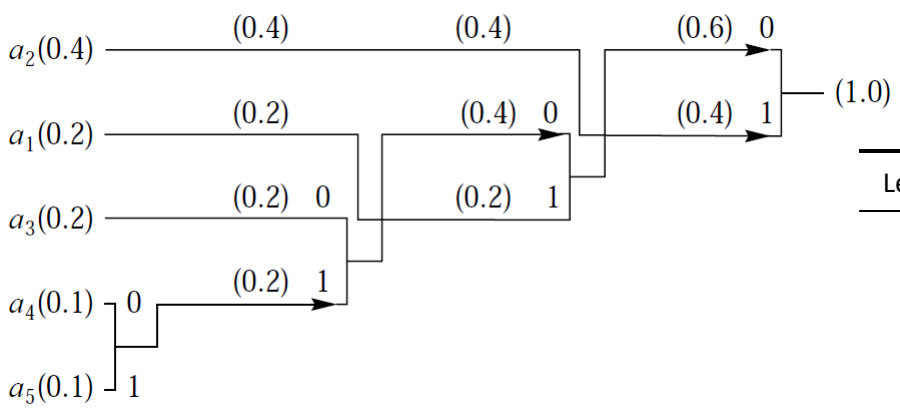
© Multimedia Technology Group

Huffman coding: Algorithm

Huffman code procedure

Example

- Designing **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$ with **probabilities** $P(a_1) = P(a_3) = 0.2$, $P(a_2) = 0.4$ and $P(a_4) = P(a_5) = 0.1$
- **Alternative** way of building **Huffman code** via **binary tree**
  - **Hufmann** code is **prefix** – **external nodes** (leaves) **correspond** to **symbols**
  - **Starting** at **leaf nodes**
  - Connecting **two leaves** corresponding **to lowest probability symbols** into **node**



Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

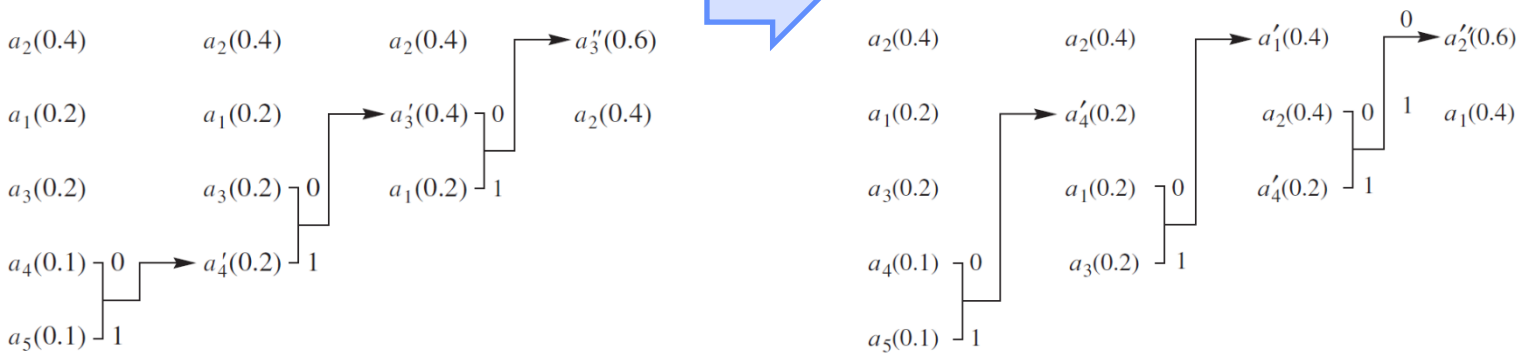
Minimum variance Huffman codes

Procedure for minimum variance Huffman codes

- Slightly **modifying sorting procedure**
  - Defining **new** (four letter) **alphabet** $\mathcal{A}' = \{a_1, a_2, a_3, a'_4\}$ where a'_4 is **composed** of a_4 and a_5 **placing as high as possible** in list

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a_1	0.2	$c(a_1)$
a_3	0.2	$c(a_3)$
a'_4	0.2	α_1

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a'_4	0.2	α_1
a_1	0.2	$c(a_1)$
a_3	0.2	$c(a_3)$



- **Combining** a_1 and a_3 into a'_1 (probability 0.4) → **sorting** and **putting** a'_1 **far up** → **combing** a_2 and a'_4 into a''_2

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

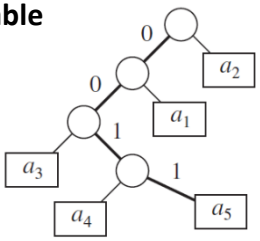
Minimum variance Huffman codes

Procedure for minimum variance Huffman codes

- Slightly **modifying sorting procedure**
  - Building **Huffman code** via **binary tree**

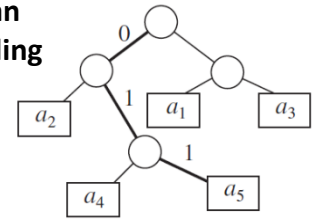
Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

Example Huffman code table and corresponding tree



Letter	Probability	Codeword
a_1	0.2	10
a_2	0.4	00
a_3	0.2	11
a_4	0.1	010
a_5	0.1	011

Minimum variance Huffman code table and corresponding tree



- **Identical average length** of **minimum variance Huffman code**
  - Related **average Huffman code length**

$$l = 0.2 \cdot 2 + 0.4 \cdot 2 + 0.2 \cdot 2 + 0.1 \cdot 3 + 0.1 \cdot 3 = 2.2 \text{ bit/symbol}$$

- **Minimum variance** Huffman codes **suitable** for **fixed rate applications**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Minimum variance Huffman codes

Procedure for minimum variance Huffman codes

- Slightly **modifying sorting procedure**
  - **Huffman code** → **variable-length** code vs. **fixed transmission** rate

Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

Example Huffman code table

Letter	Probability	Codeword
a_1	0.2	10
a_2	0.4	00
a_3	0.2	11
a_4	0.1	010
a_5	0.1	011

Minimum variance Huffman code table

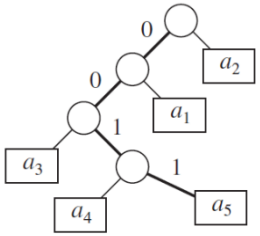
- **Example**
  - **Trasmitting symbols** 10 000 symbols/second with **average code length** 2.2 bit/symbol → **transmission capacity** 22 000 bit/second
    - **Bitrate variance** between 10 000 bps and 40 000 bps for „**example**“ **Huffman**
    - **Bitrate variance** between 20 000 bps and 30 000 bps for **minimum variance**
  - **Source coder output** fed **into** buffer → **smoothing variations** in **bitrate**
  - **Minimum variance Huffman** → **lower buffer requirements**

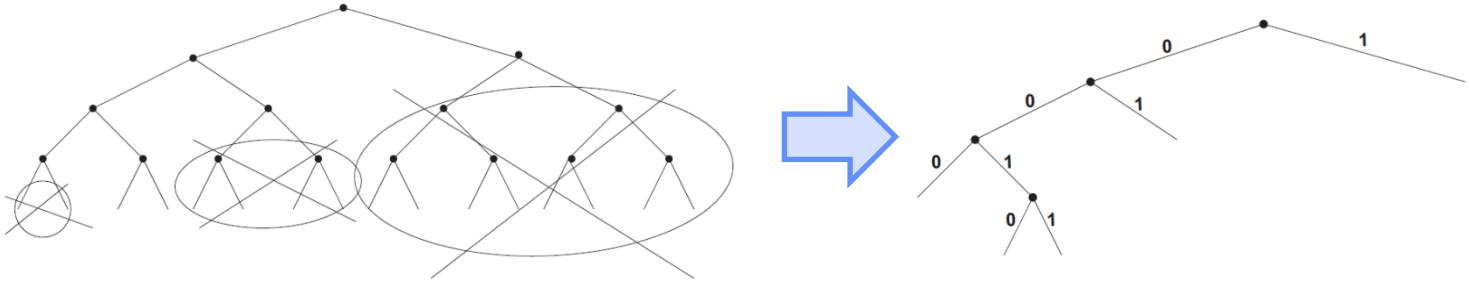
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Canonical Huffman codes

Reducing code storage requirements

- **Huffman code** reflects **source statistics**
  - **Sources** with **widely varying statistics**
  - **Huffman code** to be **included** with **compressed representation**
  - Might **cause substantial burden** (sometimes **canceling** out **compression**)
- **Canonical Huffman codes** → allows for **efficient storage**
  - Classical **Huffman** tree as **fully populated** tree → **codewords** assigned to **each leaf**
  - Given **codeword lengths** (here 3, 4, 4, 2, 1) → **code** can be **reconstructed**
  - **Pruning binary tree** (from left) → getting **codewords** of **specific length**



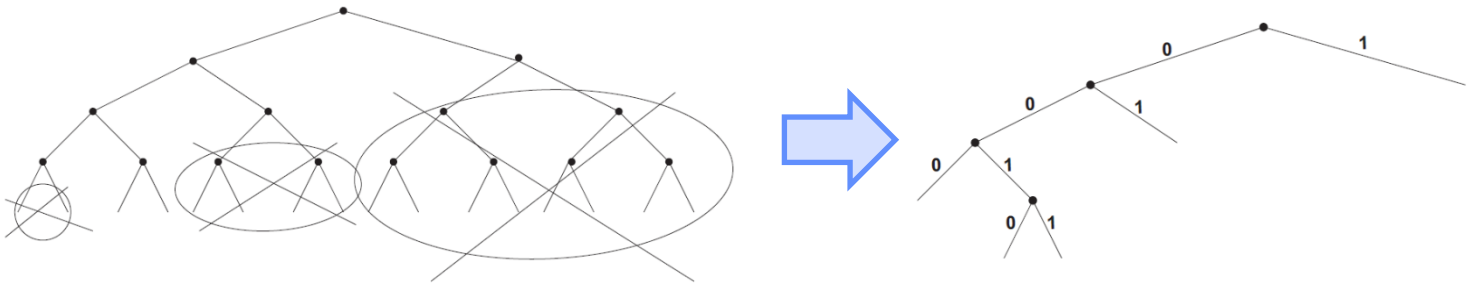


Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Canonical Huffman codes

Reducing code storage requirements

- **Canonical Huffman codes** → allows for **efficient storage**
  - Knowledge of **codewords lengths** permits **generating Huffman code**
  - **Problem** → **codeword assignment** to **symbols** (letters) **missing**
  - **Canonical procedure** → **generates code** that **contains** this **information**
- **Embedding information** → requires **additional constraints**
  - **Codes** of **given length** → **lexicographically consecutive values** with **same order** as **represented symbols**
  - **Shorter codes** → lexicographically **precede longer codes**
- **Possible conversion** of **Huffman code into canonical**



Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

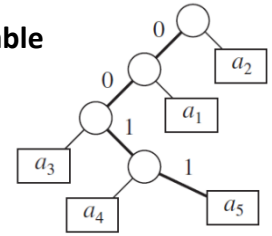
Canonical Huffman codes

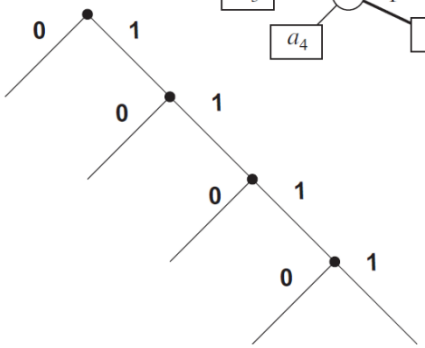
Reducing code storage requirements

- **Possible conversion** of **Huffman code into canonical**
  - Available **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$
  - Corresponding **Huffman codeword lengths** $\{2, 1, 3, 4, 4\}$
  - **Generating codewords** → from **shortest** to **longest** → **shorter codes** lexicographically **precede longer codes**
    - **(1) shortest codeword** $a_2 \rightarrow 0$
    - **(2) codeword** of **length 2** must be (1x) $a_1 \rightarrow 10$
    - **(3) codeword** of **length 3** must be (11x) $a_3 \rightarrow 110$
    - **(4) codeword** of **length 4** must be (111x) $a_4 \rightarrow 1110$ and $a_5 \rightarrow 1111$
  - **Code** can be **encoded** just **sending lengths**
- **Significant savings** for **large alphabet sizes**

Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

Example Huffman code table and corresponding tree





Canonical Huffman code tree

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Length-limited Huffman codes

- **Main goal** of **Huffman codes**
  - **Minimization** of **average codeword length**
  - There is **no limit** on **maximum length** of **individual codeword**
    - **No problem** for **limited alphabet sizes**
    - **Large alphabets** → **possibility** of getting **very long Huffman codes** → can **result** in **codeworks not fitting single machine word** → **inefficiencies** in **coding process**

  - **Adding constraint** of **maximum codeword length** l_{max}
    - For **source alphabet size** m → $l_{\max} \geq \lceil \log_2 m \rceil$ for **valid code**
    - **Kraft-McMillan inequality** → **alphabet** of size m with **codewords lengths** $\{l_1, l_2, \dots, l_m\}$ **then** for **uniquely decodable codes applies**

$$\sum_{i=1}^{m} 2^{-l_i} \leq 1$$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Length-limited Huffman codes

- **Adding constraint** of **maximum codeword length** l_{max}
  - **Kraft-McMillan inequality** → **alphabet** of size m with **codewords lengths** $\{l_1, l_2, \dots, l_m\}$ **then** for **uniquely decodable codes applies** $\sum_{i=1}^{m} 2^{-l_i} \leq 1$
  - Supposing **alphabet** of **letters** $\{a_1, a_2, \dots, a_m\}$ with **probabilities** $\{p_1, p_2, \dots, p_m\}$ **sorted** that $p_1 \leq p_2 \leq \cdots \leq p_m$
  - **Incrementing lengths** l_i until **Kraft-McMillan** is **satisfied** and **keeping average length** $\bar{l}$ as **small** as possible
    - (**1**) **assigning zero bits** to all $l_i = 0$ → $\sum_{i=1}^{m} 2^{-l_i} = \sum_{i=1}^{m} 2^0 = m$ → $m > 1$ → **Kraft-McMillan not satisfied**
    - (**2**) **increasing** $l_1 = 1$ → **Kraft-McMillan sum** $m - 0.5$ → calculating **average codeword length** $\bar{l} = \sum_{i=1}^{m} p_i l_i$
    - (**3**) **incrementing** other **codewords lengths** (reducing Kraft-McMillan sum)→ **iterative „package-merge“ algorithm**
  - **„Package-merge“** generates sorted **list of choices** (**increasing cost**)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Length-limited Huffman codes

- **Adding constraint** of **maximum codeword length** l_{max}
  - **„Package-merge“** generates sorted **list of choices** (**increasing cost**)
    - **Package step** → previous list **partitioned/packaged** (**two groups** of neighboring items) → **cost** of **package** (**sum** of **item costs** in package, **cost** of **letter** → **probability** of **letter**) → **item** with **highest cost discarded**
    - **Merge step** → previous **packages merged** with **item list** of **individual letters** → list sorted (increasing cost)
  - **After** $l_{max} - 1$ **iterations** → **lowest cost** $(2m - 2)$ items **selected**
    - **Codeword lengths** → **equal** to **number** of **items containing** that **letter**
    - For $(2m - 2)$ items → **each decrements Kraft-McMillan sum** by 0.5
    - Final set of lengths $\{l_i\}$ that $\sum_{i=1}^{m} 2^{-l_i} = 1$

- **Example**
  - Length limited **Huffman code** for **source** with **alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5, a_6\}$ with **probabilities** $P(a_1) = 0.05$, $P(a_2) = 0.1$, $P(a_3) = 0.15$, $P(a_4) = P(a_5) = 0.2$, and $P(a_6) = 0.3$
    - Related **source entropy** $H = 2.409$ bit/symbol

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Length-limited Huffman codes

Example

- Length limited **Huffman code** for **source**
  - **Source entropy** $H = 2.409$ bit/symbol
  - **Average Huffman code length** $\bar{l} = 2.45$ bits
- Added **code length constraint** $l_{max} = 3$

Letter	Probability	Codeword
a_1	0.05	0100
a_2	0.1	0101
a_3	0.15	011
a_4	0.2	10
a_5	0.2	11
a_6	0.3	00

- **(1) list** of **letters** with **costs**
  $L_0 = [a_1(0.05), a_2(0.1), a_3(0.15), a_4(0.2), a_5(0.2), a_6(0.3)]$
- **(2)** 1st **packaging step**
  $\text{Package}_1: [a_{12}(0.15), a_{34}(0.35), a_{56}(0.50)]$
- **(3)** 1st **merging step**
  $\text{Merge}_1: [a_1(0.05), a_2(0.1), a_3(0.15), a_{12}(0.15), a_4(0.2), a_5(0.2), a_6(0.3), a_{34}(0.35), a_{56}(0.50)]$
- **(4)** 2nd **packaging step** (needed to achieve **3 bits limit**)
  $\text{Package}_2: [a_{12}(0.15), a_{312}(0.30), a_{45}(0.4), a_{634}(0.65)]$
- **(5)** 2nd final **merging step**
  $\text{Merge}_2: [a_1(0.05), a_2(0.1), a_3(0.15), a_{12}(0.15), a_4(0.2), a_5(0.2), a_6(0.3), a_{312}(0.30), a_{45}(0.4), a_{634}(0.65)]$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Length-limited Huffman codes

Example

- Length limited **Huffman code** for **source**
  - **Source entropy** $H = 2.409$ bit/symbol
  - **Average Huffman code length** $\bar{l} = 2.45$ bits
- Added **code length constraint** $l_{max} = 3$
- **(5)** 2nd final **merging step**
  Merge_2: $[a_1(0.05), a_2(0.1), a_3(0.15), a_{12}(0.15), a_4(0.2), a_5(0.2), a_6(0.3), a_{312}(0.30), a_{45}(0.4), a_{634}(0.65)]$
- **(6) Picking 10 items** $(2m - 2 = 2 \cdot 6 - 2 = 10)$ with **lowest cost** and **counting number** of **letter appearance**
  - **Codeword lengths** $\{3, 3, 3, 3, 2, 2\}$ → a_1 appears in a_1, a_{12}, a_{312}, …

Letter	Probability	Codeword
a_1	0.05	0100
a_2	0.1	0101
a_3	0.15	011
a_4	0.2	10
a_5	0.2	11
a_6	0.3	00

Huffman code

- Length limited **Huffman code**
  - **Average length-limited Huffman code length** $\bar{l} = 2.5$ bits
  - **Cost** of **limiting length** to $l_{max} = 3$ → $2.5 - 2.45 = 0.05$ bits

Letter	Probability	Codeword
a_1	0.05	100
a_2	0.1	101
a_3	0.15	110
a_4	0.2	111
a_5	0.2	00
a_6	0.3	01

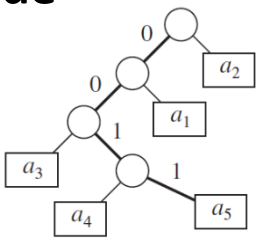
Length-limited Huffman code

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Optimality of Huffman codes

Proving optimality of Huffman codes

- **Necessary conditions** for **optimal variable-length binary code**
  - (**1**) Given **two letters** a_j and a_k, **probabilities** $P(a_j) \geq P(a_k)$, **then** $l_j \leq l_k$, where l_j is **codeword length** of a_j
  - (**2**) Two **least probable letters** with **same maximum length** l_m **codewords**
  - (**3**) In **optimum code tree** there must be **two branches stemming** from **each intermediate node**
    - **Intermediate node** with only **one branch** → **can** be **removed without affecting decodability** while **reducing** its **average length**
  - (**4**) **Changing intermediate node** (in optimal tree) into **leaf node** (**combining** all **descending leaves** into **composite word**) → **reduced tree** also **optimal** for **reduced alphabet**



- **Satisfying** (**1**, **2**, **3**)
  - **Two least probable letters** with **same maximum length** l_m **codewords**
  - These **codewords identical except** for **last bit**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Length of Huffman codes

- **Huffman coding procedure** generates **optimum code**
  - **Average length** of **optimum code** depends on
    - **Size** of **alphabet**
    - **Letter** (symbol) **probabilities**
  - **Optimum code** (Huffman code) for **source** $\mathcal{S}$ has **average code length** $\bar{l}$ **bounded** by **entropy** $H(\mathcal{S})$ and $H(\mathcal{S})+1$

$$H(\mathcal{S}) \leq \bar{l} < H(\mathcal{S}) + 1$$

    - More specifically **Huffman code rate** bounds
      - $H(\mathcal{S}) \leq \bar{l} < H(\mathcal{S}) + p_{max} + 0.086$ for $p_{max} < 0.5$
      - $H(\mathcal{S}) \leq \bar{l} < H(\mathcal{S}) + p_{max}$ for $p_{max} \geq 0.5$
      - For **most frequently occurring symbol** p_{max}
  - **Related** to **Kraft-McMillan inequality** → **alphabet** of size m with **codewords lengths** $\{l_1, l_2, \dots, l_m\}$ **then** for **uniquely decodable codes applies** $\sum_{i=1}^{m} 2^{-l_i} \leq 1$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Implementation of Huffman codes

- **Huffman encoding**
  - Relatively **simple** to **implement** → using **lookup table**
- **Huffman decoding**
  - **More complex problem**
    - For **high speed** → also using **lookup table** → table of **size** 2^N (N – **longest codeword**)
      - Significant amount of **wasted storage**
      - Waste **grows exponentially** with **longest codeword**
    - For **memory efficiency** → **decoding** → **traversing binary tree**
      - **Might** be **computationally inefficient**
    - **Combining aspects**
      - **Hybrid table/tree** based **decoder**
      - **Decoder** can be **viewed** as **state machine**
      - **State machine** → takes **specified number** of **input bits** → may be **different** for **each state**

Letter	Code
A	0
B	10101
C	11
D	101000
E	10100101
F	1010011
G	100
H	10100100
I	1011

Nine-letter alphabet and Huffman code

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

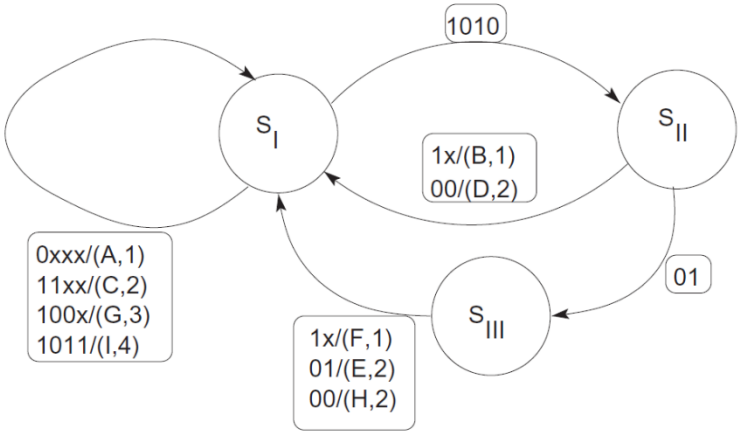
© Multimedia Technology Group

Implementation of Huffman codes

Huffman decoding

State machine hybrid method

- **State machine** → takes **specified number** of **input bits** → may be **different** for **each state**
- **State machine** → having **received** those **bits** → machine **moves** to **new state** → outputs **decoded letter**



State machine description of Huffman decoding

Example

- **Tabular implementation** requires **table size 256**
- **Hierarchy** of **3 tables** (**size 16**, **4**, and **4**)
- **16 values points** to
  - **Decoded letter** and **number** of **bits** used to **decode** or
  - **Subsidiary table**
- **Decoding represented** with **3 state machine**

Letter	Code
A	0
B	10101
C	11
D	101000
E	10100101
F	1010011
G	100
H	10100100
I	1011

Nine-letter alphabet and Huffman code

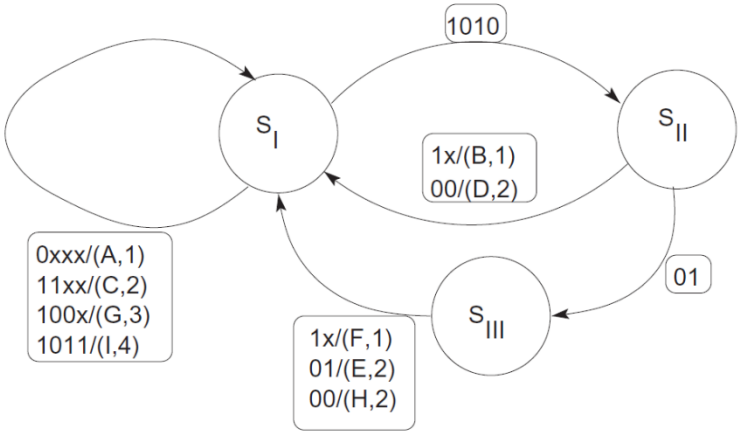
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Implementation of Huffman codes

Huffman decoding

Example

- **Decoding represented** with **3 States machine**
  - S_I → **4-bit input** as **index** to **Table I**
  - $S_{II, III}$ → **2-bit input** as **index** to **Table I, II**
- **Decoder moves** S_I → S_{II} for (1010)
- **Decoder moves** S_{II} → S_{III} for (01)
- **Other patterns** $nnnn/(L, m)$
  - $nnnn$ – **binary index**
  - L – **decoded letter**
  - m – **number** of **bits used**
  - x – **do not care** $(0$ or $1)$



State machine description of Huffman decoding

Letter	Code
A	0
B	10101
C	11
D	101000
E	10100101
F	1010011
G	100
H	10100100
I	1011

Nine-letter alphabet and Huffman code

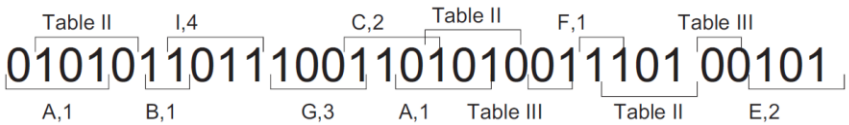
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Implementation of Huffman codes

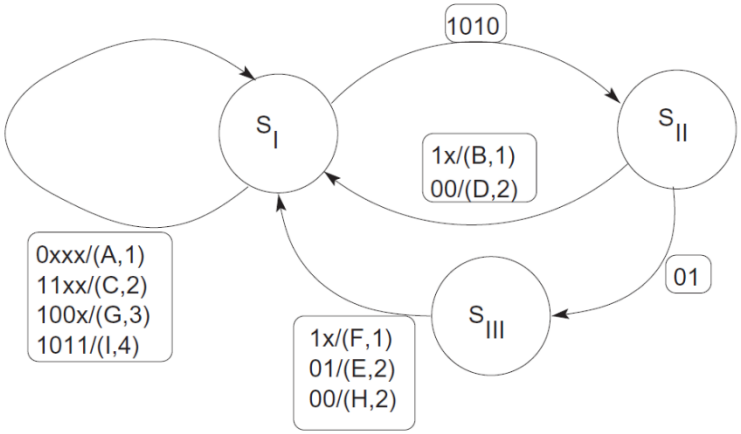
Huffman decoding

Example

- **Decoding represented** with **3 States machine**
  - **S_I → 4-bit input** as **index** to **Table I**
  - **$S_{II, III}$ → 2-bit input** as **index** to **Table II, III**
- **Decoding bitstream**



  - **(1) Starting** in **S_I** (0101)
  - **(2) Decoding** using **Table I** → $(A, 1)$
  - **(3) Shifting 1 bit** → (1010) → **S_{II}** (10)
  - **(4) Decoding** using **Table II** → $(B, 1)$, …
- **Only 1.0472 lookups** per **letter**



State machine description of Huffman decoding

Code	Letter
0000	$A, 1$
0001	$A, 1$
0010	$A, 1$
0011	$A, 1$
0100	$A, 1$
0101	$A, 1$
0110	$A, 1$
0111	$A, 1$
1000	$G, 3$
1001	$G, 3$
1010	Table II
1011	$I, 4$
1100	$C, 2$
1101	$C, 2$
1110	$C, 2$
1111	$C, 2$

Table I

Code	Letter
00	$D, 2$
01	Table III
10	$B, 1$
11	$B, 1$

Table II

Code	Letter
00	$H, 2$
01	$E, 2$
10	$F, 1$
11	$F, 1$

Table III

Letter	Code
A	0
B	10101
C	11
D	101000
E	10100101
F	1010011
G	100
H	10100100
I	1011

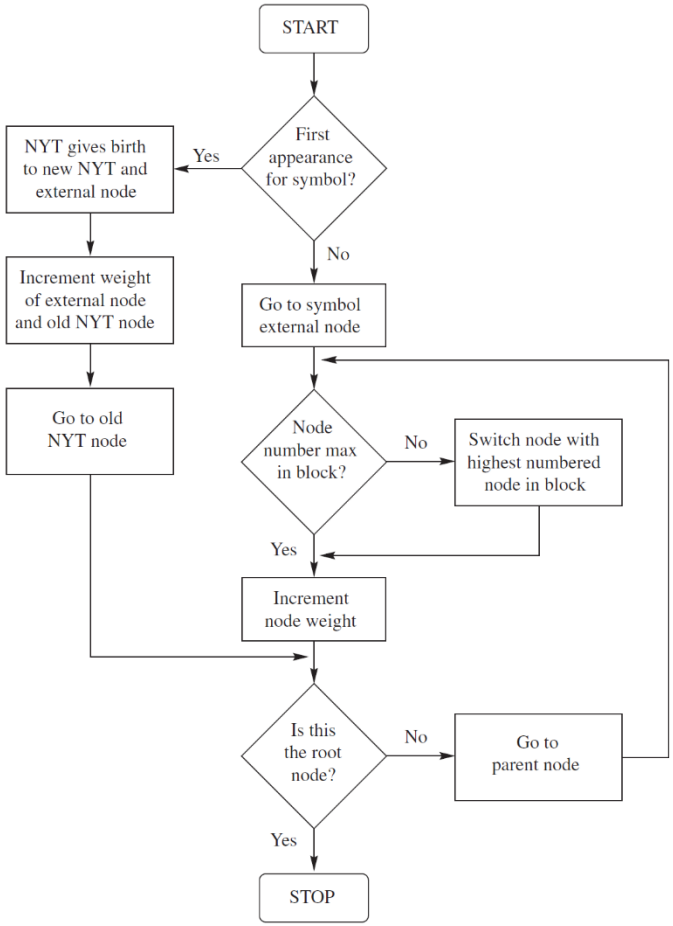
Nine-letter alphabet and Huffman code

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Adaptive Huffman coding

Traditional Huffman coding

- **Requires knowledge** of **source sequence probabilities**
- **Probabilities unknown**
  - **Traditional Huffman coding** becomes **two-pass procedure**
    - **(1) Statistics collected** in **first pass**
    - **(2) Source encoded** in **second pass**
  - **Adaptive Huffman coding**
    - Allows **one-pass procedure** → **constructing Huffman code** based on **statistics** of already **encountered symbols**
    - **Theoretically statistics** for **coding** $(k+1)$th **symbol** can be based on first k **symbols** → **requires recomputing** of **Huffman procedure** for **every symbol**
    - **Adaptive Huffman coding** → **allows** for **more efficient procedure**



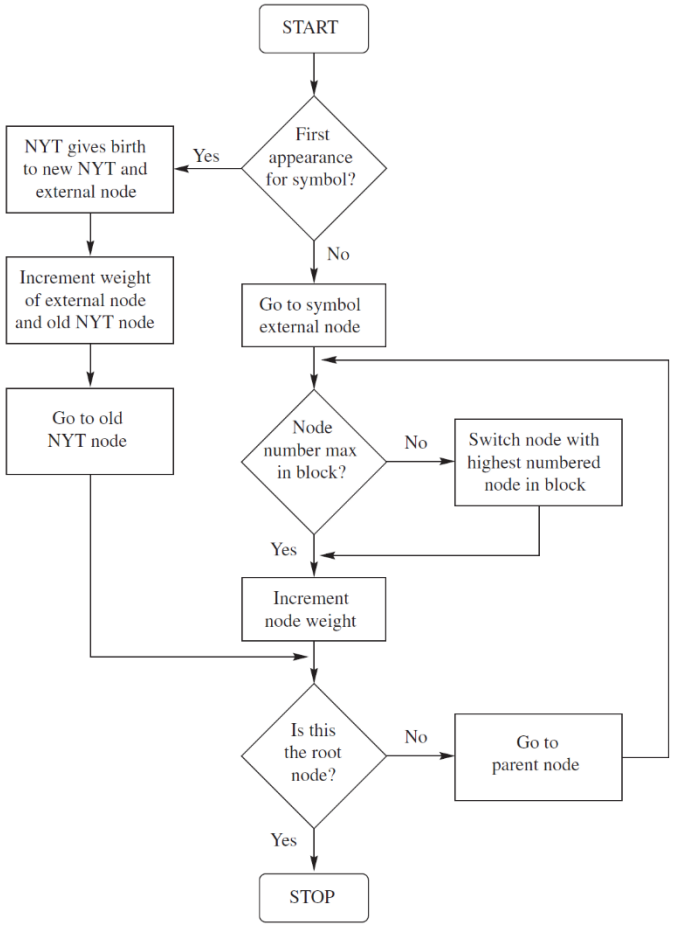
Update procedure for adaptive Huffman coding

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Adaptive Huffman coding

Adaptive Huffman coding

- **Huffman code described** using **binary tree**
  - **Two parameters** added to **binary tree**
    - (1) **weight** of each **leaf** (number inside node)
    - (2) **node number** (unique number assigned)
- **Transmitter** and **receiver**
  - **No statistics known** before **transmission**
    - (1) **Fixed code** for **each symbol agreed**
    - (2) **Tree** consists of **single node NYT** (not yet transmitted) with **zero weight**
    - (3) When **symbol encountered first time** → **NYT code transmitted** followed by **fixed symbol code** → **node created** → symbol **taken from NYT list**
    - (4) Transmission **progressing** → **nodes added** → **tree reconfigured** using **update procedure**
  - **Starting with same structure** → **updates**



Update procedure for adaptive Huffman coding

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Golomb codes

- **Huffman coding**
  - One of **best known variable length coding** algorithms
  - Other **codes** (**Golomb-Rice**, **Tunstall**) **increasingly popular**
- **Golomb-Rice codes**
  - **Codes designed** to **encode integers**
  - Assuming **larger integer** → **lower probability** of **occurrence**
  - **Unary code**
    - **Positive integer** n → simply n $1s$ **followed** by 0
      - **Code** for 4 → 11110, **code** for 7 → 11111110, …
      - **Unary code same** as **Huffman code** for **semi-infinite alphabet** $\{1, 2, 3, \ldots\}$ and **probability model** $P[k] = 1/2^k$ →
      - **Unary code optimal** for this **probability model**
      - **Very restricted conditions** but **simple** to **implement**
    - Coding **schemes splitting integers** into **two parts** (**Unary** + **Other**)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Golomb codes

- **Golomb code** (Solomon Golomb, 1966)
  - **Family** of **codes parametrized** by **integer** $m > 0$
    - **Golomb code** with **parameter** m **represents integer** $n > 0$ using **two numbers** q and r
      - where $q = \left\lfloor \frac{n}{m} \right\rfloor$ (integer part of) → **quotient** when n **divided** by m
      - and $r = n - q\,m$ → **remainder** when n **divided** by m
    - **Quotient** q takes **values** $(0, 1, 2, \dots)$ **represented** in **unary code** of q
    - **Remainder** r takes **values** $(0, 1, 2, \dots, m-1)$
      - For m **power** of **two** → $\log_2 m$ – bit **binary representation** of r
      - For m **not power** of **two** → $\lceil \log_2 m \rceil$ bits (smallest integer grater or equal to)
    - **Required number** of **bits reduction**
      - **Remainder** r represented $\lfloor \log_2 m \rfloor$ **bits** for **first** $(2^{\lceil \log_2 m \rceil} - m)$ **values** and $\lceil \log_2 m \rceil$ **bits** representation of $(r + 2^{\lceil \log_2 m \rceil} - m)$ for **rest** of **values**
    - **Golomb code** is **optimal** for **probability model** $P(n) = p^{n-1} q$ where $q = 1 - p$ **when** $m = \left\lceil -\frac{1}{\log_2 p} \right\rceil$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Golomb codes

Golomb code

Example

- Designing **Golomb code** for $m = 5$
- (**1**) **Calculating** $\lceil \log_2 5 \rceil = 3$ and $\lfloor \log_2 5 \rfloor = 2$
- (**2**) **First** $(2^{\lceil \log_2 m \rceil} - m) = 2^3 - 5 = 3$ **values** of **residual** r $(r = 0, 1, 2)$ represented **using 2-bit binary**
- (**3**) **Next** 2 **values** of **residual** r will be **represented** as $(r + 2^{\lceil \log_2 m \rceil} - m) = r + 8 - 5 = r + 3$ by $\lceil \log_2 m \rceil = \lceil \log_2 5 \rceil = 3$ **bits**
- (**4**) **Quotient** q represented by **unary code**

- **Codeword** for $3 \rightarrow (0110)$, …
- **Example** of **Golomb code** for $n = 0, \ldots, 15$ and $m = 5$

Golomb code for $m = 5$

n	q	r	Codeword	n	q	r	Codeword
0	0	0	000	8	1	3	10110
1	0	1	001	9	1	4	10111
2	0	2	010	10	2	0	11000
3	0	3	0110	11	2	1	11001
4	0	4	0111	12	2	2	11010
5	1	0	1000	13	2	3	110110
6	1	1	1001	14	2	4	110111
7	1	2	1010	15	3	0	111000

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Rice codes

- **Rice code** (Robert Rice, 1979)
  - Can be **viewed** as **adaptive Golomb code**
    - **Sequence** of **nonnegative integers** divided into **blocks** of J **integers**
    - Each **block coded** using **several options** → **Golomb codes**
    - **Option** resulting in **least number** of **coded bits selected**
    - **Option indicated** by **identifier attached** to **code** of each **block**
  - **Example CCSDS**
    - **Recommendation** (Consultative Committee on Space Data Standards) for **lossless compression**
    - **Algorithm contains two phases**
      - **Preprocessor** (modeling step) → **removes correlation** → generates **sequence** of **nonnegative integers** → **smaller values** more **probable** than **larger**
      - **Binary coder** (coding step) → generates **bitstream** to represent **integer sequence**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Rice codes

Rice code

Example CCSDS

- **Preprocessor function**
  - For **given sequence** $\{y_i\}$ generates **prediction** $\hat{y}_i$ → **simplest prediction** based on **previous value** $\hat{y}_i = y_{i-1}$
  - Generates **difference sequence** $d_i = y_i - \hat{y}_i$ (ideally **small** d_i **values**)
  - For **largest** y_{max} and **smallest** y_{min} **values assuming** $\hat{y}_i$ **confined** $[y_{min}, y_{max}]$ then $T_i = \min\{y_{max} - \hat{y}_i, \hat{y}_i - y_{min}\}$
  - **Sequence** $\{d_i\}$ **converted** in **nonnegative integers** $\{x_i\}$

$$x_i = \begin{cases} 2d_i & 0 \le d_i \le T_i \\ 2|d_i| - 1 & -T_i \le d_i \le 0 \\ T_i + |d_i| & \text{otherwise} \end{cases}$$

  - Value x_i **small** with **higher probability**
  - **Sequence** $\{x_i\}$ **divided** into **segments** and **segments** into **blocks** of **size** J
  - Value $J = 16$ **recommended** by **CCSDS**
  - **Each block coded** using **one option**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Rice codes

Rice code

Example CCSDS

- **Encoder function**
  - **Each block coded** using **one option** while **identifier** is **transmitted**
  - **(a) Fundamental sequence**
    - **Unary code** → n → simply n $1s$ **followed** by **0**
  - **(b) Split sample options**
    - **Set** of **codes** indexed by **parameter** m → consists of m **least significant bits** followed by **unary code** representing $(k-m)$ **most significant bits**
    - **Number** 23 → 00010111 → **3 LSBs** → 111 → **remaining bits** (00010) → number 2 → unary code (110) → **final code** (111110)
  - **(c) Second extension option**
    - **Useful** for **sequences** with **low entropy** → **many zero** x_i values
    - **Sequence divided** into **pair** of **samples** → receiving **index** $\gamma = \frac{1}{2}(x_i + x_{i+1})(x_i + x_{i+1} + 1) + x_{i+1}$ → **unary code** → **pointer** to **lookup table**
  - **(d) Zero block option** (coding table)
    - For **one** or **more blocks** of x_i are **zero** (same y_i)

Number of All-Zero Block	Codeword
1	1
2	01
3	001
4	0001
5	000001
6	0000001
⋮	⋮
63	$\overbrace{000\cdots0}^{630s}1$
ROS	00001

Zero block option code

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Tunstall codes

- **Tunstall codes** (Brian Parker Tunstall, PhD thesis 1967)
  - Most of **variable-length codes** → **varying number** of codeword **bits** (fewer bits for frequent letters…)
  - **Tunstall code** has **codewords** of **equal length**
    - **Each codeword** represents **different number** of **letters**
    - Main **advantage** → **errors** in codewords **do not propagate**
  - **Example**
    - Encoding **sequence** $AAABAABAABAABAAA$
      - Starting at left using **2-bit Tunstall code** table
      - Getting **code** $AAA\ B\ AAB\ AAB\ AAB\ AAA$ → $00\ 11\ 01\ 01\ 01\ 00$

Sequence	Codeword
AAA	00
AAB	01
AB	10
B	11

2-bit Tunstall code

  - **Tunstall code** should **satisfy conditions** → we **should**
    - (**1**) be **able** to **parse source output sequence** into **codebook symbols**
    - (**2**) **maximize average number** of **source symbols** represented by each **codeword**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Tunstall codes

Tunstall codes

- **Tunstall code** should **satisfy conditions** → we **should**
  - (**1**) be **able** to **parse source output sequence** into **codebook symbols**
  - (**2**) **maximize average number** of **source symbols** represented by each **codeword**
  - **Algorithm fulfilling** these **conditions**
    - (**1**) n-bit **Tunstall code** for source generating **letters** from N-size **alphabet** → **number** of **codewords** 2^n
    - (**2**) **Starting** with N **letters** in **codebook** → **remove** entry with **highest probability** → add **concatenating this** letter with **every letter** in **alphabet** (including itself) → **increasing codebook** size from $N \rightarrow N + (N - 1)$
    - (**3**) Find **highest probability letter** (products of probabilities) among $N + (N - 1)$ → add **concatenating this** with **every letter**
    - (**4**) Each step **increasing codebook size** by $(N - 1)$
    - **Operation** can be **performed** K times where $N + K(N - 1) \leq 2^n$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Tunstall codes

Tunstall codes

Example

- Designing **3-bit Tunstall code** with **alphabet** $\mathcal{A} = \{A, B, C\}$ and **probabilities** $P(A) = 0.6$, $P(B) = 0.3$ and $P(C) = 0.1$
- **Procedure**
  - (**1**) **Removing** A with **highest probability**
  - (**2**) **Adding two-letter strings** beginning with A → **increasing codebook** size from $N = 3 \rightarrow N + (N - 1) = 5$ (still **less** than **codebook size** of 8)
  - (**3**) **Removing** AA with **highest probability** → **adding concatenated sequences** beginning with AA → **increasing codebook** size from $N = 3 \rightarrow N + 2(N - 1) = 7$ (still **less** than **codebook size** of 8)
  - (**4**) Another iteration would increase codebook size $N = 3 \rightarrow N + 3(N - 1) = 9$ (**more** than **codebook size** of 8)
  - **Obtaining** final **3-bit Tunstall code**

Letter	Probability
A	0.60
B	0.30
C	0.10

Source alphabet and probabilities

Sequence	Probability
B	0.30
C	0.10
AA	0.36
AB	0.18
AC	0.06

Codebook after one iteration

Sequence	Code
B	000
C	001
AB	010
AC	011
AAA	100
AAB	101
AAC	110

3-bit Tunstall code

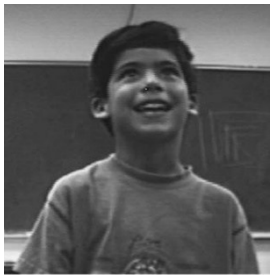
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

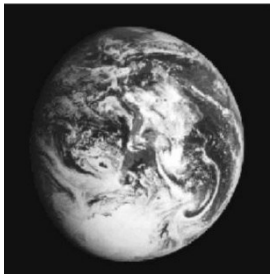
© Multimedia Technology Group

Examples of Huffman coding

Simple lossless image compression

- **Example compression** for **8bpp images**
  - Intensity **values** $0..255$
  - **Images** of 256×256 **pixels**
  - → resulting in $65\,536$ **bytes uncompressed**





Compression using Huffman codes on pixel values

Bits/Pixel	Total Size (bytes)	Compression Ratio
7.01	57,504	1.14
7.49	61,430	1.07
4.94	40,534	1.62
7.12	58,374	1.12

Compression using Huffman codes on pixel DIFFERENCE values

Bits/Pixel	Total Size (bytes)	Compression Ratio
4.02	32,968	1.99
4.70	38,541	1.70
4.13	33,880	1.93
6.42	52,643	1.24

Compression using ADAPTIVE Huffman codes on pixel DIFFERENCE values

Bits/Pixel	Total Size (bytes)	Compression Ratio
3.93	32,261	2.03
4.63	37,896	1.73
4.82	39,504	1.66
6.39	52,321	1.25

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Examples of Huffman coding

Simple lossless audio compression

Example compression for „CD-audio quality“

- Sample **bit depth 16 bps** (65 536 levels) **per channel** (**stereo**)
- **Sampling frequency** of 44.1 kHz → bitrate ~1.4 Mbps
- **Three** different **musical recordings** (symphonic, folk rock, …)

Huffman coding of 16-bit CD audio

File Name	Original File Size (bytes)	Entropy (bits)	Estimated Compressed File Size (bytes)	Compression Ratio
Mozart	939,862	12.8	725,420	1.30
Cohn	402,442	13.8	349,300	1.15
Mir	884,020	13.7	759,540	1.16

Huffman coding of DIFFERENCE 16-bit CD audio

File Name	Original File Size (bytes)	Entropy of Differences (bits)	Estimated Compressed File Size (bytes)	Compression Ratio
Mozart	939,862	9.7	569,792	1.65
Cohn	402,442	10.4	261,590	1.54
Mir	884,020	10.9	602,240	1.47

- Very **simple modeling used** here → **sample** has **same value** as **previous**
  - Usable **lossless audio compression schemes** → more **sophisticated linear predictive coding**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

References and recommended materials

- [1] Sayood, K.: **Introduction to Data Compression**, Morgan Kaufmann Publishers, 2006.
- [2] Sayood K.: **Lossless Compression Handbook**, Elsevier, 2003.
- [3] Kahrs, M., Brandenburg, K.: **Applications of Digital Signal Processing to Audio and Acoustics**, Kluwer Academic Publishers, 1998.
- [4] Gonzalez, R. C., Woods, R. E.: **Digital Image Processing**, Pearson, 2007.
- [5] Salomon, D., Motta, G.: **Handbook of Data Compression**, Springer-Verlag London, 2010.
- [6] Salomon, D.: **A Concise Introduction to Data Compression**, Springer, 2008.
- [7] Thyagarajan, K. S.: **Still Image and Video Compression with MATLAB**, Willey, 2011.

Thank you for your attention!

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

Technická 2
166 27 Prague 6
Czech Republic

© Multimedia Technology Group